# 1. 文件操作的重要性和应用场景

## 1.1 重要性

- **数据持久化**：文件是存储数据的一种非常基本且重要的方式。通过文件，我们可以将程序运行时产生的数据永久保存下来，以便将来使用。
- **跨平台兼容性**：文件是一种通用的数据交换格式，可以在不同的操作系统和平台上进行传输和处理。
- **数据备份与恢复**：定期备份数据到文件有助于防止数据丢失，便于数据恢复。
- **数据共享**：文件可以轻松地在网络之间共享，使得多人协作成为可能。
- **配置管理**：许多应用程序使用文件来存储配置信息，方便用户根据需要调整设置。
- **日志记录**：文件被广泛用于记录程序运行时的日志，这对于调试和性能监控非常重要。

## 1.2 应用场景

- **数据分析**：读取数据文件进行数据分析，或者程序的运行结果输出到文件，以便于报告或进一步分析。
- **Web开发**：读取配置文件来设置Web服务器的各种参数，处理用户上传的文件。
- **系统管理**：读取和写入日志文件以监控系统状态。
- **文本处理**：读取文本文件，进行搜索、替换等操作，并将处理后的结果保存到文件中。
- **游戏开发**：读取账号数据、保存账号数据，管理游戏资源文件。

华清远见元宇宙实验中心 yyzlab.com.cn

# 2. 文件的基本概念

## 2.1 文件的概念

文件是一个存储在某种持久性存储介质（如硬盘、固态驱动器或磁带等）上的数据集合。文件可以包含各种类型的信息，包括文本、图像、音频、视频、应用程序代码以及其他类型的二进制数据。文件是操作系统用来组织和管理这些数据的主要方式之一。

文件通常由以下几部分组成：

华清远见 HQYJ.COM

- **数据**：文件中存储的实际信息，即用户想要保存的具体信息，如文本、图像或代码等。
- **元数据**：关于文件本身的附加信息，包括但不限于文件名、创建日期、文件大小、文件类型等。
- **文件系统**：这是操作系统用来组织和管理文件的一种逻辑结构，包括文件的命名、存储和检索方式。文件系统还负责管理磁盘空间的分配，并确保文件可以被正确地读写。常见的文件系统有 FAT32、NTFS 等。

## 2.2 文件的属性

1. **文件名**：用于标识文件的唯一名称，通常包含主文件名和扩展名（如 `document.txt`）。
2. **位置**：文件存储在一个特定的位置，这个位置可以用目录或路径来表示。例如，在Windows系统中，文件路径可能是 `C:\Documents\example.txt`；而在Unix-like系统中，路径可能是 `/home/user/Documents/example.txt`。
3. **文件类型**：根据文件的内容和用途，文件可以有多种类型，如文本文件、图像文件、音频文件等。
4. **文件大小**：文件占用的存储空间大小，通常以字节（B）、千字节（KB）、兆字节（MB）或吉字节（GB）为单位。
5. **创建日期和时间**：文件被创建的时间戳。
6. **修改日期和时间**：文件最后一次被修改的时间戳。
7. **访问权限**：定义了不同用户对文件的读、写、执行权限。

## 2.3 文件的类型

在Windows系统中，大致可以分为以下几种：

1. **文本文件**：包含可读字符的文件，如.txt、.csv、.html等。
2. **二进制文件**：包含不可直接读的原始二进制数据的文件，如.exe、.jpg、.mp3等。
3. **可执行文件**：可以被操作系统执行的文件，如.exe（Windows）。
4. **数据文件**：用于存储应用程序数据的文件，如数据库文件、配置文件等。
5. **目录/文件夹**：用于组织和管理其他文件的特殊文件。

在Linux系统中，可以分为以下几种：

- **-**：普通文件，比如txt、py等。
- **d**：目录文件，比如xx目录，类比Windows中的文件夹。
- **b**：块设备文件，Linux系统中的底层驱动文件。

华清远见开发实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

- **c**：字符设备文件，Linux系统中的底层驱动文件。
- **l**：链接文件，类似于快捷方式
- **p**：管道文件，用于进程间的通信。
- **s**：套接字文件，用于网络通信的端点，用于网络传输。

# 2.4 文件的路径

在计算机文件系统中，路径是用来标识文件或目录位置的一种方式。路径有两种主要的形式：绝对路径和相对路径。这两种路径形式对于在文件系统中导航和访问文件非常重要。

## 2.4.1 绝对路径

绝对路径是从文件系统的根目录开始的一条完整路径，它指明了从根目录到达目标文件或目录的具体步骤。

**特点**：

- 不依赖于当前工作目录。
- 在不同用户或程序间具有一致性。
- 提供了文件或目录的完整位置信息。

**示例**：

- 在Windows系统中，一个绝对路径可能看起来像这样：`C:\Users\John\Documents\report.txt`。
- 在Linux/Unix系统中，一个绝对路径可能看起来像这样：`/home/john/Documents/report.txt`。

## 2.4.2 相对路径

相对路径是指相对于某个起始点（通常是当前工作目录）到达目标文件或目录的路径。

**特点**：

- 取决于当前工作目录的位置。
- 更加灵活，但可能会因上下文变化而变化。
- 适用于在同一目录层级或附近层级内的文件访问。

**示例**：

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

- 在Windows系统中，如果当前工作目录是 `C:\Users\John\Documents`，那么我想去找同目录下的example.txt的话，就是.\example.txt，如果上级目录下的example.txt的话，就是..\example.txt。
- 在Linux/Unix中，如果当前工作目录是 `/home/john/Documents`，那么我想去找同目录下的example.txt的话，就是./example.txt，如果上级目录下的example.txt的话，就是../example.txt。

### 2.4.3 路径中的特殊符合

- `.` **(当前目录)**：表示当前目录。
- `..` **(上一级目录)**：表示当前目录的父目录。

### 2.4.4 使用场景

- **绝对路径**：当需要指定确切位置，或者在不同环境（如不同用户的系统）下保持一致时使用。
- **相对路径**：当文件位于同一目录或相关联的子目录中时使用，可以使程序更加灵活和便携。

## 2.5 文件系统简介

文件系统是一种逻辑结构，它定义了如何在物理存储设备（如硬盘驱动器、固态硬盘等）上组织和管理文件。文件系统负责跟踪文件的位置、大小、元数据（如创建时间、修改时间等），并提供创建、读取、更新和删除文件的接口，方便操作系统或用户进行文件的操作。

其主要功能为：

1. **命名**：为文件和目录提供唯一的名称。
2. **存储**：管理文件在存储设备上的物理位置。
3. **检索**：允许用户通过文件名或其他属性查找文件。
4. **更新**：支持文件的创建、修改和删除。
5. **权限管理**：控制用户和组对文件和目录的访问权限。
6. **磁盘空间管理**：跟踪可用和已用磁盘空间，并在必要时进行空间分配和回收。
7. **错误恢复**：在发生错误（如系统崩溃或电源故障）时，尝试恢复文件系统的完整性。

常见的文件系统类型包括：

- **FAT (File Allocation Table)**：主要用于较旧的系统或移动存储设备。

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

- **NTFS (New Technology File System)**：Windows 操作系统的默认文件系统。
- **HFS+ (Hierarchical File System Plus)**：以前是 macOS 的默认文件系统。
- **APFS (Apple File System)**：macOS 和 iOS 设备的新一代文件系统。
- **ext4 (Fourth Extended File System)**：Linux 操作系统的常用文件系统。
- **XFS (XFS File System)**：另一种用于 Linux 的文件系统。

# 3. 文件操作的基本方法

## 3.1 打开文件

在Python中，使用**open()**函数来打开文件，这个函数返回一个文件对象，可以用来进行后续的读写等操作。

函数原型为：

```
res = open(file_name, mode='r', buffering=None, encoding=None, errors=None, newline=None, closefd=True)
```

- **file_name**：要打开的文件的路径加名称（包含后缀名），可以是绝对路径也可以是相对路径。
- **mode**：打开文件的模式，默认为'r'，表示只读模式且以文本模式读取。
- **buffering**：可选参数，缓冲区大小。0表示无缓冲，1表示行缓冲，更大的整数表示具体的缓冲区大小。默认为None，表示默认的缓冲策略，大多数情况下，使用默认值就可以了。
- **encoding**：可选参数，用于指定文件的编码，仅适用于文本模式，默认值None表示使用系统的默认编码来打开文本文件。
- **errors**：可选参数，用于指定如何处理编码和解码错误，对于二进制模式无效。常见的值有**strict**、**ignore**、**replace**等。
- **newline**：可选参数，用于控制通用换行符模式的行为。它可以是`None`、`''`、`'\n'`、`'\r'`或`'\r\n'`。如果设置为`None`，则通用换行符模式被启用，`\n`、`\r`和`\r\n`都被识别为换行符，并以`\n`的形式在文本模式下读取。如果设置为其他值，则在该值处进行换行符的转换。
- **closefd**：可选参数，如果为`True`（默认值），则在文件关闭时关闭文件描述符。如果为`False`，则文件描述符在文件关闭时保持打开状态。

## 3.2 文件模式

以下是几种常见的文件打开模式：

华清远见嵌入式实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

- `'r'`：read 只读模式（默认值）。如果文件不存在就会触发异常。
- `'r+'`：打开文件进行读写，该文件必须存在。
- `'w'`：write 写入模式，如果文件存在则覆盖，不存在则创建。
- `'w+'`：打开文件进行读写，如果文件存在则覆盖，如果不存在则创建。
- `'a'`：追加模式，如果文件存在则在文件末尾追加内容，不存在则创建。
- `'a+'`：打开文件进行读写，如果文件存在则在末尾追加，如果不存在则创建。
- `'x'`：独占创建模式，如果文件已存在则抛出异常，这可以用来避免覆盖现有文件。
- `'b'`：二进制模式，读写时，数据不会被转换，直接以字节形式处理。
- `'t'`：文本模式（默认值），读写时，数据会被视为字符串。

## 3.3 读取文件

打开文件后，可以使用以下方法读取内容：

- `read(size)`：size是可选参数，在文本模式下，一次最多读取文件指针后面size个大小的字符，在二进制模式下，一次最多读取文件指针后面size个大小的字节，默认size为None，表示一次性读取文件指针后面的所有内容并将其作为字符串返回。
- `readline()`：从文件中读取单行数据。
- `readlines()`：读取所有行，并返回一个列表。

## 3.4 写入文件

要将内容写入文件，可以使用以下方法：

- `write(str)`：将str的内容覆盖到当前文件指针位置的后面，并将文件指针移动到新的写入位置。会返回写入的字符数量，写入其他类型的对象时，要先将它们转化为字符串或字节对象。
- `writelines()`：写入一个字符串列表。

## 3.5 关闭文件

在python中，使用**close()**方法关闭文件。关闭文件是一个重要的操作，因为它释放了与文件对象关联的系统资源，并确保数据正确地写入存储介质。

**功能**

1. **释放资源**：关闭文件，释放与文件对象关联的所有系统资源，如文件描述符、缓冲区等。

华清远见嵌入式实验中心 www.yzlab.com.cn

华清远见 HQYJ.COM

2. **刷新缓冲区**：在关闭文件之前，它会自动刷新文件的内部缓冲区，确保所有缓冲的数据都被写入磁盘。
3. **禁止进一步操作**：关闭文件后，文件对象不再允许进行读取、写入或其他操作。

**重要性**

文件操作完成后，应该关闭文件以释放资源。可以使用**close()**方法关闭文件。在Python程序中，如果不关闭打开的文件，可能会产生以下影响：

1. **资源管理**：文件描述符是有限的资源，如果不关闭文件，可能会导致资源泄漏，特别是在打开大量文件时。
2. **数据完整性**：确保所有缓冲的数据都写入磁盘，防止数据丢失。尤其是在写入操作后，如果不关闭文件，可能会导致最后写入的数据没有保存。
3. **防止错误**：关闭文件可以防止对已关闭文件的非法操作，这些操作可能会引发异常。
4. **提高效率**：关闭不再需要的文件可以释放系统资源，提高程序的整体效率。
5. **清理操作**：在关闭文件时，可以执行一些清理操作，如关闭网络连接或释放其他相关资源。

# 3.6 with语句

在Python中，**with**语句是一种上下文管理器（context manager），用于简化资源的打开和关闭过程，确保资源在不需要时得到适当的释放。这种机制常用于文件操作、网络连接、锁等资源的处理，可以避免资源泄露和出现其他资源管理问题。

基本语法为：

```
with expression [as variable]:
    with-block
```

1. **表达式（expression）**：这个表达式必须返回一个实现了上下文管理器协议的对象，也就是说，它需要包含`__enter__`和`__exit__`两个方法。
2. `as`**子句**：这是可选的。如果提供了`as`子句，那么`expression`中`__enter__`方法的返回值将被赋值给变量。
3. `with-block`：这个代码块是`with`语句的主体，在执行这个代码块之前，会首先调用上下文管理器的`__enter__`方法。当`with-block`执行完毕后，不论是因为正常完成还是因为异常，都会调用上下文管理器的`__exit__`方法，该方法负责关闭文件。

执行流程为：

华清远见宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

1. **执行表达式**：首先执行 `expression`，返回一个上下文管理器对象。
2. **调用 `__enter__()` 方法**：上下文管理器对象的 `__enter__()` 方法被调用，该方法通常用于执行一些初始化操作，如打开文件。
3. **执行语句块**：语句块中的语句按照顺序执行。
4. **调用 `__exit__()` 方法**：当语句块执行完毕后，上下文管理器对象的 `__exit__()` 方法被调用，该方法通常用于执行清理操作，如关闭文件或释放数据库会话。

**使用 `with` 语句的好处**

- **自动资源管理**：不需要手动调用 `close` 方法，代码更简洁。
- **更好的错误处理**：`__exit__()` 方法可以包含异常处理代码，确保在发生异常时资源得到妥善处理。
- **更好的代码组织**：`with` 语句有助于将相关的操作组织在一起，使代码更加清晰。

# 4. 文件指针的操作

## 4.1 获取文件指针的位置

在python中，使用**tell()**函数去返回当前文件指针的位置。

```
tell()
```

tell()函数没有参数，它的功能就是返回文件指针当前位置相对于文件开头的偏移量。这个偏移量是一个整数，表示从文件开头到当前读取位置的**字节数**。

**注意事项**

- `tell` 方法仅在文件被打开用于读取时才有意义，因为在写入模式下，文件指针的位置会随着写入操作而改变。
- 在读取模式下，`tell` 方法返回的是当前读取位置相对于文件开头的偏移量。
- 其返回值是字节数，不是字符数，对于utf-8的编码格式来说，一个汉字占三个字节，所以读取中文时，字符数与字节数是不一样的。

## 4.2 改变文件指针的位置

在python中，使用**seek()**函数去改变文件指针的位置。

```
seek(offset, whence=0)
```

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

其中，

**offset**：表示相对于whence的偏移量，是一个整数。这个偏移量可以是正数，也可以是负数。正数表示向文件末尾方向移动，负数表示向文件开头方向移动，0则表示不偏移。

**whence**：是一个可选的参数，默认为0。它指定了**offset**的起始位置，可以是以下三个值之一：

- `0`：表示从文件开头开始计算偏移量（默认值）。
- `1`：表示从当前文件指针位置开始计算偏移量。
- `2`：表示从文件末尾开始计算偏移量。

**注意事项**

- `seek` 方法在文本模式和二进制模式下都有效，但文本模式whence只能使用默认值，不能自己修改。
- `seek` 方法基于字节偏移量。这意味着即使文件包含多字节字符，`seek` 改变的仍然是字节偏移量。

# 5. 获取文件属性

在Python中，可以使用内置的**os**模块来获取文件的属性。以下是一些常用的方法来获取文件属性：

## 5.1 获取文件大小

在python中，使用os.path.getsize()获取指定文件的大小（以字节为单位）。

```
os.path.getsize(path)
```

- `path` (字符串) - 文件的路径。

返回一个整数，表示文件的大小（以字节为单位）。如果文件不存在或无法访问，会抛出异常。

## 5.2 获取文件的最后修改时间

在python中，使用os.path.getmtime()获取指定文件的最后修改时间。

```
os.path.getmtime(path)
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

- `path` (字符串) - 文件的路径。

返回一个浮点数，表示文件的最后修改时间，自纪元（1970年1月1日 00:00:00 UTC）以来的秒数，可以搭配time模块进行格式化。如果文件不存在或无法访问，会抛出异常。

## 5.3 获取文件的创建时间

在python中，使用os.path.getctime()获取指定文件的创建时间。

```
os.path.getctime(path)
```

- `path` (字符串) - 文件的路径。

返回一个浮点数，表示文件的最后修改时间，自纪元（1970年1月1日 00:00:00 UTC）以来的秒数，可以搭配time模块进行格式化。如果文件不存在或无法访问，会抛出异常。

## 5.4 获取文件的最后访问时间

在python中，使用os.path.getatime()获取指定文件的最后访问时间。

```
os.path.getatime(path)
```

- `path` (字符串) - 文件的路径。

返回一个浮点数，表示文件的最后修改时间，自纪元（1970年1月1日 00:00:00 UTC）以来的秒数，可以搭配time模块进行格式化。如果文件不存在或无法访问，会抛出异常。

华清远见|元宇宙实验中心 yyzlab.com.cn

# 6. 目录操作

在Python中，可以使用多个模块来操作目录，但是最常用的就是os模块，以下就是一些常见的目录操作：

## 6.1 创建目录

在Python中使用**os.mkdir(path)**函数来创建目录。

```
import os
os.mkdir(path, mode=0o777)
```

华清远见 HQYJ.COM

- `path` (字符串) - 要创建的目录的路径。
- `mode` (整型，可选) - 设置新创建目录的权限位。默认值是 `0o777` （八进制表示），意味着所有人都有读、写和执行权限，只针对Linux系统，Windows系统会忽略。

如果目录创建成功，则函数不返回任何内容。如果指定的路径已经存在就会抛出异常，如果路径是无效的，或者由于权限不足等原因无法创建目录，也会抛出异常。

**注意事项**

- `os.mkdir` 只能创建一级目录，如果父目录不存在，则会抛出异常。
- 如果需要创建多级目录结构，可以使用 `os.makedirs` 函数，它会递归地创建所需的中间目录。

# 6.2 删除目录

在Python中使用**os.rmdir(path)**函数去删除目录。

```
import os
os.rmdir(path)
```

- `path` (字符串) - 要删除的空目录的路径。

如果目录删除成功，则函数不返回任何内容。如果指定的路径不存在，则会抛出异常，如果路径不是一个空目录，或者由于权限不足等原因无法删除目录，也会抛出此异常。

**注意事项**

- `os.rmdir` 只能删除空目录。如果目录中包含文件或其他目录，`os.rmdir` 将无法删除它，并且会抛出异常。

# 6.3 改变当前工作目录

在Python中使用**os.chdir(path)**函数来改变工作目录。

```
import os
os.chdir(path)
```

- `path` (字符串) - 要切换到的目录的路径。如果指定的路径不存在、指定的路径不是一个目录、没有权限更改到指定的目录就会抛出异常。

华清远见元宇宙实验中心 vyzlab.com.cn

华清远见 HQYJ.COM

如果目录切换成功，则函数不返回任何内容。

## 6.4 获取当前工作目录

在Python中使用**os.getcwd()**函数获取当前的工作目录。

```
os.getcwd()
```

该函数没有参数，但会返回一个字符串，表示当前工作目录的路径。

**注意事项**

1. **os.getcwd()** 返回的是字符串形式的路径。
2. 在不同的操作系统中，路径的表示方式可能不同。例如，在 Windows 中路径通常使用反斜杠 `\`，而在 Unix/Linux 系统中使用正斜杠 `/`。

这个函数常用于以下场景：

- 当需要在脚本中确定当前的工作位置时。
- 在进行文件操作之前，需要知道文件的相对路径是基于哪个目录。
- 当需要在不同目录之间切换时，但之后想要回到原始目录。

## 6.5 列出目录下的所有内容

在Python中使用**os.listdir(path)**函数获取指定目录下的所有文件和子目录的名称。

```
import os
items = os.listdir(path)
```

- `path` (字符串) - 要列出内容的目录的路径。如果省略，默认为当前工作目录。如果指定的路径不存在、指定的路径不是一个目录、没有权限读取指定的目录就会抛出异常。

这个函数返回一个列表，其中包含指定路径下的所有文件和子目录的名称。

**注意事项**

- `os.listdir` 不会递归地列出子目录中的内容。它只列出直接位于指定目录下的文件和子目录。
- 返回的列表中只包含名称，不包含路径。如果需要完整路径，你需要将目录名称与路径结合起来。
- 如果目录为空，返回的列表将是空的。

华清远见|云学用实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

- 在使用 `os.listdir` 时，应该考虑到可能出现的异常，并适当处理它们，以确保代码的健壮性。

# 6.6 重命名目录

在Python中使用**os.rename(src，dst)**函数对一个文件或目录进行重命名。这个函数可以将一个文件或目录从其当前路径（源路径）更改为一个新的路径（目标路径）。

```
import os
os.rename(src, dst)
```

- `src` (字符串) - 要重命名的文件或目录的当前路径。
- `dst` (字符串) - 文件或目录的新名称和路径。

如果重命名成功，则函数不返回任何内容。

# 6.7 检查路径是否为目录

在Python中使用**os.path.isdir()**函数检查给定的路径是否是一个目录。

```
import os.path
is_directory = os.path.isdir(path)
```

- `path` (字符串) - 要检查的路径。

如果指定的路径是一个目录，返回 `True`；否则返回 `False`。

**注意事项**

- `os.path.isdir` 不会抛出异常。如果指定的路径不存在，它会返回 `False`，而不是抛出异常。
- 如果路径存在但不是目录，`os.path.isdir` 也会返回 `False`。
- 路径可以是绝对路径，也可以是相对路径。

# 4.8 检查路径是否为文件

在Python中使用**os.path.isfile()**函数检查指定的路径是否是一个存在的文件。

```
import os
is_file = os.path.isfile(path)
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

- `path` (字符串) - 要检查的路径。

如果指定的路径是一个文件，返回 `True`；否则返回 `False`。

**os.path.isfile()** 的主要用途包括：

1. 在处理文件之前，确认指定的路径确实是一个文件，这样可以避免对目录执行文件操作。
2. 在脚本中执行条件逻辑时，根据路径是否为文件来决定下一步操作。
3. 在文件处理脚本中，区分文件和目录，以便进行适当的操作。

需要注意的是，**os.path.isfile()** 不会检查路径是否存在，如果路径不存在，它将直接返回 `False`。如果需要同时检查路径是否存在以及是否为文件，可以先使用 **os.path.exists()** 确认路径存在，然后再使用 **os.path.isfile()** 确认它是一个文件。

# 4.8 路径拼接

在Python中使用**os.path.join(path, *path)**函数将一个或多个路径组件合并成一个完整的路径。这个函数会根据操作系统的文件系统约定来正确地处理路径分隔符。

```
import os
os.path.join(path, *paths)
```

- `path` (字符串) - 起始路径，通常是一个目录路径。
- `*paths` (可变参数) - 需要连接到 `path` 的其他路径片段。

返回一个字符串，表示将所有路径片段连接后的完整路径。

**os.path.join()**的作用包括：

1. **合并路径**：将多个路径组件合并成一个单一的路径字符串。
2. **处理分隔符**：它会根据操作系统自动添加或删除路径分隔符（例如，在 Windows 上是反斜杠 `\`，在 Unix/Linux 上是正斜杠 `/`）。
3. **消除冗余分隔符**：如果路径组件之间有多余的分隔符，**os.path.join()** 会自动处理，避免产生错误的路径。

使用**os.path.join()**的好处是，它能够确保生成的路径在不同的操作系统上是正确的，从而提高了代码的可移植性。此外，它还能避免手动拼接路径时可能出现的错误，如忘记添加分隔符或添加了错误的分隔符。

# 4.9 路径拆分

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

在Python中使用**os.path.split(path)**函数将路径分割成两部分：目录路径和文件名。

```
import os
head, tail = os.path.split(path)
```

- `path`：表示要分割的路径。
- `head`：它是 `path` 的目录路径
- `tail`：它是 `path` 的文件名

**os.path.split()**的主要用途包括：

1. 从完整路径中提取文件名或目录名。
2. 用于文件处理时，需要单独操作文件名和路径的其他部分。
3. 在遍历文件系统时，帮助确定每个文件的上级目录。

需要注意的是，如果提供的路径以路径分隔符结尾，那么 `tail` 将是一个空字符串，表示路径的最后一部分是一个目录。此外，如果路径是空字符串，**os.path.split()**将返回两个空字符串。

# 4.10 获取绝对路径

在Python中使用**os.path.abspath(path)**函数将相对路径转换为绝对路径。

```
import os
absolute_path = os.path.abspath(path)
```

- `path` (字符串) - 要转换为绝对路径的相对路径或文件路径。

返回一个字符串，表示转换后的绝对路径。

**os.path.abspath()** 的主要用途包括：

1. 在脚本中确保使用的是文件的绝对路径，这样即使当前工作目录改变，脚本也能正确地定位文件。
2. 将相对路径转换为绝对路径，以便在不同的环境中更可靠地引用文件或目录。
3. 在需要输出或记录文件的完整位置时使用。

需要注意的是，如果提供的 `path` 是一个不存在于文件系统中的路径，**os.path.abspath()** 仍然会返回一个绝对路径，但它不会验证路径是否存在。此外，如果 `path` 是一个空字符串，**os.path.abspath()** 将返回当前工作目录的绝对路径。

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

# 4.11 检查路径是否存在

在Python中使用**os.path.exists(path)**函数检查指定的路径（文件或目录）在文件系统中是否存在。

```
import os
path_exists = os.path.exists(path)
```

- `path`：表示要检查的路径。
- `path_exists`：返回一个布尔值，如果路径存在则返回 `True`，如果路径不存在则返回 `False`。

**os.path.exists()** 的主要用途包括：

1. 在执行文件操作之前，确认文件或目录是否存在，以避免引发不必要的错误。
2. 在脚本中执行条件逻辑时，根据文件或目录的存在与否来决定下一步操作。
3. 在文件备份、清理或其他自动化任务中，检查特定文件或目录是否应该被处理。

需要注意的是，**os.path.exists()** 仅仅检查路径是否存在，而不会区分该路径是文件还是目录。如果需要明确区分文件和目录，可以使用 **os.path.isfile()** 或 **os.path.isdir()**。

华清远见元宇宙实验中心 yyzlab.com.cn